 ll Vs Agile – Comparison Table

Feature	Waterfall Model	Agile Model
Approach	Linear and sequential	Iterative and incremental
Process Flow	Moves step-by-step (Requirements → Design → Development → Testing → Deployment)	Continuous cycles (Plan → Develop → Test → Review → Repeat)
Flexibility	Rigid, changes are difficult once a phase is completed	Highly flexible; changes can be made anytime
Requirements	Must be clearly defined at the beginning	Can evolve throughout the project
Customer Involvement	Low; client reviews mainly at the end	High; client provides continuous feedback in every iteration
Delivery	Final product delivered after full development	Working increments delivered frequently (e.g., every Sprint)
Testing	Done after development is complete	Continuous testing within each iteration
Team Collaboration	Limited; roles are clearly defined and follow a structured hierarchy	High collaboration; cross-functional teams work together
Documentation	Extensive and mandatory at each stage	Minimal documentation; focuses on working software
Cost & Time	Higher cost if changes occur; timeline fixed	Cost-effective for changing requirements; timeline adaptive

Creating an Amazon Application Using the Waterfall Model

The **Waterfall Model** is a linear, sequential software development methodology.
Each phase must be completed **fully** before the next begins.
No going back.

The Waterfall phases (as shown in your image) are:

1. **Requirements Gathering**
2. **Design**
3. **Implementation**
4. **Testing**
5. **Deployment**
6. **Maintenance**

Below is the detailed step-by-step process for building an **Amazon online shopping application** using these phases.

1. Requirements Gathering Phase

In this phase, the project team meets with stakeholders (business owners, customers, admin teams, logistics teams) to gather all functional and non-functional requirements.

A. Functional Requirements (specify what the system should do) **of an Amazon App**

1. **User Management**
   - User registration, login, logout
   - User profile editing
   - Address management
   - Wishlist creation
2. **Product Catalog**
   - Categories & subcategories
   - Product search
   - Filters (brand, price, rating, delivery options)

1. Requirements Gathering Phase

In this phase, the project team meets with stakeholders (business owners, customers, admin teams, logistics teams) to gather all functional and non-functional requirements.

A. Functional Requirements (specify what the system should do) **of an Amazon App**

1. **User Management**
    - User registration, login, logout
    - User profile editing
    - Address management
    - Wishlist creation
2. **Product Catalog**
    - Categories & subcategories
    - Product search
    - Filters (brand, price, rating, delivery options)
    - Product comparison
    - Product detail pages with images, reviews, stock status
3. **Shopping Cart**
    - Add/remove products
    - Quantity updates
    - Price calculation
4. **Order Management**
    - Order placement
    - Real-time order tracking
    - Order history
5. **Payment System**
    - Credit/debit card, UPI, net banking, EMI
    - Secure payment gateway integration
6. **Delivery & Logistics**
    - Delivery address selection
    - Delivery date estimation
    - Return/refund system
7. **Admin Portal**
    - Add/edit/delete products
    - Inventory updates
    - Review moderation
    - Analytics dashboard

B. Non-Functional Requirements

- Security (SSL, data encryption, authentication)
- Scalability (handling millions of users)
- Performance (fast search, quick load)
- Reliability (99.9% uptime)
- Usability (simple and clean UI)

Output of this phase:

✓ SRS (Software Requirements Specification) document
✓ Project scope and timeline
✓ Approval from stakeholders

2. Design Phase

Based on the SRS, architects and designers prepare the **system blue**

A. High-Level Design (HLD)

- Security (SSL, data encryption, authentication)
- Scalability (handling millions of users)
- Performance (fast search, quick load)
- Reliability (99.9% uptime)
- Usability (simple and clean UI)

Output of this phase:

✓ SRS (Software Requirements Specification) document
✓ Project scope and timeline
✓ Approval from stakeholders

2. Design Phase

Based on the SRS, architects and designers prepare the **system blueprint**.

A. High-Level Design (HLD)

Defines the system architecture:

- **Frontend Architecture**
  - Web & Mobile app screens
  - UI flow diagram
  - Navigation structure
- **Backend Architecture**
  - Microservices or monolithic
  - Database design (ER diagram)
  - APIs required
- **Modules**
  - Product module
  - Cart module
  - Orders module
  - Payment module

B. Low-Level Design (LLD)

Details for developers:

- Class diagrams
- Database table structures
- Data flow diagrams
- API endpoint designs
- Business logic flow

Output of this phase:

✓ System Architecture Document
✓ UI mockups

✓ Database design diagrams
✓ LLD document

3. Implementation Phase (Coding)

Developers start writing the code based on LLD.

Frontend Development

- Build responsive UI for web/mobile
- Login system
- Product listing and product detail pages
- Cart & checkout pages

Backend Development

- Build REST APIs
- Implement business logic

✓ Database design diagrams
✓ LLD document

3. Implementation Phase (Coding)

Developers start writing the code based on LLD.

Frontend Development

- Build responsive UI for web/mobile
- Login system
- Product listing and product detail pages
- Cart & checkout pages

Backend Development

- Build REST APIs
- Implement business logic
- Set up authentication & authorization
- Integrate payment gateway
- Build order processing system

Database Development

- Create schemas and tables
- Write stored procedures
- Implement indexing for fast search

Tech Stack (Example)

- **Frontend:** React / Angular / Flutter
- **Backend:** Java / Node.js / Python
- **Database:** MySQL / PostgreSQL / MongoDB
- **Hosting:** AWS (EC2, S3, RDS)

Output of this phase:

✓ Complete source code
✓ Integration of all modules

4. Testing Phase

Testers verify that the application meets all the requirements defined in the SRS.

Types of Testing

1. **Unit Testing** – testing each module
2. **Integration Testing** – combining modules
3. **System Testing** – e-commerce workflow testing
4. **UI/UX Testing** – layout, navigation
5. **Performance Testing -**
   - Handling huge product lists
   - Concurrent orders
6. **Security Testing**

a. SQL injection
b. Payment protection
c. OTP protection

7. **User Acceptance Testing (UAT)**
   a. Real users test the system

Output of this phase:

✓ Test case documents
✓ Bug reports & fixes
✓ Final approval for deployment

5. Deployment Phase

The application is moved from development to production servers.

Activities

- Deploy backend services on cloud (AWS EC2)
- Deploy frontend (AWS S3 / CloudFront)
- Configure load balancers
- Connect payment gateway in live mode
- Setup domain (DNS configuration)

Output of this phase:

✓ Live Amazon-like application
✓ Available to real users

6. Maintenance Phase

After launch, continuous updates and bug fixes occur.

Maintenance Activities

- Fix bugs reported by customers
- Add new features (e.g., "Buy Again", "Same-day delivery")
- Improve security
- Update product catalog UI
- Handle peak traffic during sales

Examples of Enhancements

- Introducing Amazon Pay
- Adding voice-based search
- Personalized recommendations

Output of this phase:

✓ Stable application
✓ Regular updates
✓ Improved customer satisfaction

✓ Improved customer satisfaction

Phase	What Happens?	Key Deliverables
Requirements	Collect needs & expectations	SRS
Design	Architecture + UI + DB design	HLD, LLD
Implementation	Coding backend & frontend	Source Code
Testing	Find & fix defects	Test Reports
Deployment	Release to users	Live App
Maintenance	Updates & support	Patches, new features

Creating an Amazon Application Using Agile Methodology

Agile is an **iterative and incremental** development approach.
Instead of completing the entire project in one long cycle (like Waterfall), Agile delivers the product in **small, usable increments (Sprints)**.

Each Sprint (2–4 weeks) goes through these phases:

1. **Planning**
2. **Requirement Analysis**
3. **Designing**
4. **Implementation**
5. **Testing**
6. **Deployment**
7. **Review & Feedback → next Sprint**

This cycle continues until the full Amazon application is built.

1. Planning (Sprint Planning)

In Agile, planning happens **before every Sprint**, not just once.

Activities

- Product Owner prioritizes features in the **Product Backlog**.
- Team selects the highest-value tasks for the **Sprint Backlog**.
- Define **Sprint Goal** (e.g., "Implement product search feature").
- Break features into small user stories.

Example Stories for an Amazon App

- "As a user, I want to search for products by name."
- "As a customer, I want to add products to the cart."
- "As an admin, I want to upload new products."

Output

Output

✓ Sprint backlog
✓ User stories + acceptance criteria
✓ Defined sprint duration

2. Requirement Analysis

Agile focuses on **small sets of requirements per Sprint**.

Activities

- Clarify user stories in detail
- Create acceptance criteria
- Discuss edge cases with developers & testers
- Prepare sample data / product attributes

Example for Amazon Application

For **Checkout Feature** in a Sprint:

- Payment methods to include (UPI, cards, net banking)
- Address selection flow
- Delivery date estimation logic

Output

✓ User stories with clear requirements
✓ Prioritized backlog items
✓ Story points (effort estimation)

3. Designing (UI/Architecture for the Sprint)

Unlike Waterfall, Agile design is **incremental**.

Activities

- Design only the features included in the Sprint
- UI/UX team builds quick wireframes
- Developers design API flows for the story
- Update architecture as new features emerge

Examples

Sprint 1:

- Create homepage wireframe
- Product listing card layout

Sprint 2:

- Add product detail page design

Sprint 3:

- Create checkout flow design

Output

✓ Updated design documents
✓ UI mockups for new features
✓ Updated architecture diagrams

4. Implementation (Coding for the Sprint)

Developers build the features selected for the Sprint.

Activities

- Write code for frontend features
- Create or update backend APIs
- Integrate databases
- Follow coding standards
- Do daily stand-ups

✓ Updated design documents
✓ UI mockups for new features
✓ Updated architecture diagrams

4. Implementation (Coding for the Sprint)

Developers build the features selected for the Sprint.

Activities

- Write code for frontend features
- Create or update backend APIs
- Integrate databases
- Follow coding standards
- Do daily stand-ups

Example for Amazon-like App

Sprint Task: "User can add items to cart"

- Build **Add to cart** API
- Implement cart UI
- Show item count on top of the page

Output

✓ Working software increments
✓ API endpoints
✓ Updated database with necessary tables

5. Testing (Continuous Testing)

Testing happens **during the Sprint**, not after all modules are built.

Types of Testing in Agile

- Unit testing
- Functional testing
- Integration testing
- API testing
- Regression testing within the Sprint
- Automated testing (if possible)

Example

For "Product Search" Sprint:

- Test search results accuracy
- Test filter options
- Test load time & performance

Output

Output

✓ Passed/failed test cases
✓ Bugs logged into tracking system
✓ Fixes done within the same sprint

6. Deployment (Incremental Release)

At the end of each sprint:

- Features are deployed to a **staging environment**
- Sometimes deployed to **production** (Continuous Deployment)
- AWS tools like **EC2, S3, CloudFront, Lambda** may be used

Example

Sprint 2 deployed features:

- Product details page
- Customer reviews module

Output

✓ Working version for stakeholders
✓ New features available to test users

7. Review, Feedback & Retrospective

This is unique to Agile.

A. Sprint Review

- Demonstrate the working Amazon features
- Stakeholders provide feedback
- Features can be modified in the next sprint

B. Retrospective

Team discusses:

- What went well?
- What could be improved?
- What processes should change?

Example Feedback

- Add voice search to product list
- Improve cart page loading speed

Output

✓ Updated backlog
✓ Process improvements
✓ Enhanced feature plan

REWORK: SCRUM & JIRA FRAMEWORK

Scrum

Scrum is an Agile framework used for developing complex products through collaboration, iterative progress, and continuous improvement. It organizes work into short, time-boxed cycles called *sprints* (usually 2–4 weeks), during which a cross-functional team delivers usable increments of the product. Scrum includes

✓ Updated backlog

✓ Process improvements

lan

REWORK: SCRUM & JIRA FRAMEWORK

Scrum

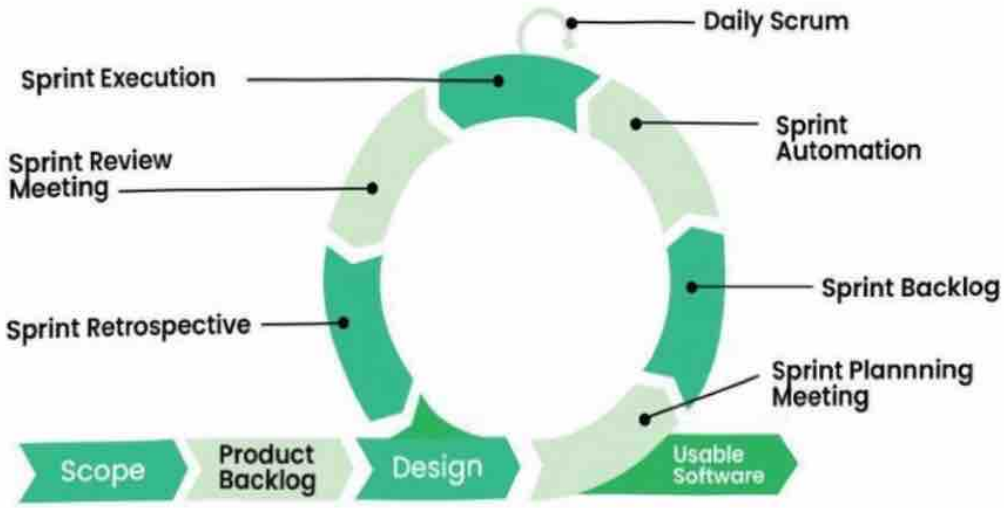
Scrum is an Agile framework used for developing complex products through collaboration, iterative progress, and continuous improvement. It organizes work into short, time-boxed cycles called *sprints* (usually 2–4 weeks), during which a cross-functional team delivers small, usable increments of the product. Scrum includes defined roles (Product Owner, Scrum Master, Development Team), key events (Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective), and artifacts (Product Backlog, Sprint Backlog, Increment) that help teams stay focused, respond quickly to change, and continuously improve both the product and the process.

Three Pillars of Scrum

1. **Transparency:** Everyone should clearly see the work and process.
2. **Inspection:** The team regularly checks progress and the product.
3. **Adaptation:** The team adjusts and improves based on what they learn.

Scrum Roles

1. **Product Owner:**
   Represents the customer, decides what features are important, and manages the product backlog.
2. **Scrum Master:**
   Supports the team, removes obstacles, and ensures Scrum rules are followed.
3. **Development Team:**
   Builds and delivers a working piece of software at the end of each sprint.



2. **Daily Scrum (Daily Stand-up):**
   A short daily meeting where team members share progress and plans.
3. **Sprint Review:**
   The team shows the finished work to customers and stakeholders for feedback.
4. **Sprint Retrospective:**
   The team discusses what went well and what needs improvement for the next sprint.

Features of Scrum

1. **Iterative and Incremental:**
   Work is done in short cycles called sprints (usually 2–4 weeks), and each sprint delivers a usable product part.
2. **Clear Roles:**
   Scrum has defined roles—Product Owner, Scrum Master, and Development Team—to keep responsibilities clear.
3. **Artifacts:**
   Scrum uses **Product Backlog**, **Sprint Backlog**, and **Increment** to organize work and show progress.
4. **Ceremonies for Collaboration:**
   Scrum meetings (planning, daily standup, review, retrospective) help the team stay aligned and transparent.

What is Jira?

Jira is a project and product management tool developed by Atlassian. It helps teams plan, track, and manage their work in an organized way. Using Jira, teams can create tasks, assign them, set deadlines, and follow progress. It is widely used in software development, especially in Agile environments.

Benefits of Jira Agile

- **Better Collaboration:** Team members can see each other's tasks and work together easily.
- **Flexible to Changes:** Jira Agile allows teams to change priorities and adjust work quickly.
- **Easy Task Management:** Creating, assigning, and tracking tasks becomes simple.
- **Project Transparency:** Dashboards and reports show clear progress to everyone.
- **Customizable:** Teams can create templates and workflows based on their needs.
- **Real-Time Tracking:** You can monitor progress instantly and make fast decisions.
- **Tool Integration:** Works well with many third-party tools to improve efficiency.

Advantages of Jira

- **Issue Tracking:** Easily create, track, and manage issues or tasks.
- **Integrations:** Connects with many tools like Git, CI/CD tools, and version control systems.
- **Custom Fields:** Allows adding custom fields to collect specific information.
- **Search & Reporting:** Provides powerful search filters and detailed reports.

- **Agile Support:** Supports Scrum and Kanban with boards, sprints, and backlog management.

Disadvantages of Jira

- **Complex Setup:** Can be difficult for beginners to configure and customize.
- **Learning Curve:** New users may take time to understand how Jira works.
- **Cost:** Becomes expensive as the team size increases.
- **Performance Issues:** May slow down with many users or large data.
- **Administrative Overhead:** Requires regular maintenance and management effort.

When to Choose Jira?

- **Software Development:** To plan work, execute tasks, and manage changing requirements.
- **Scrum Projects:** Useful for managing backlogs, sprints, and sprint planning.
- **Kanban Projects:** Helps visualize work using Kanban boards and improve workflow.

Rework 2 day 5

- **Custom Fields:** Allows adding custom fields to collect specific information.
- **Search & Reporting:** Provides powerful search filters and detailed reports.

- **Agile Support:** Supports Scrum and Kanban with boards, sprints, and backlog management.

Disadvantages of Jira

- **Complex Setup:** Can be difficult for beginners to configure and customize.
- **Learning Curve:** New users may take time to understand how Jira works.
- **Cost:** Becomes expensive as the team size increases.
- **Performance Issues:** May slow down with many users or large data.
- **Administrative Overhead:** Requires regular maintenance and management effort.

When to Choose Jira?

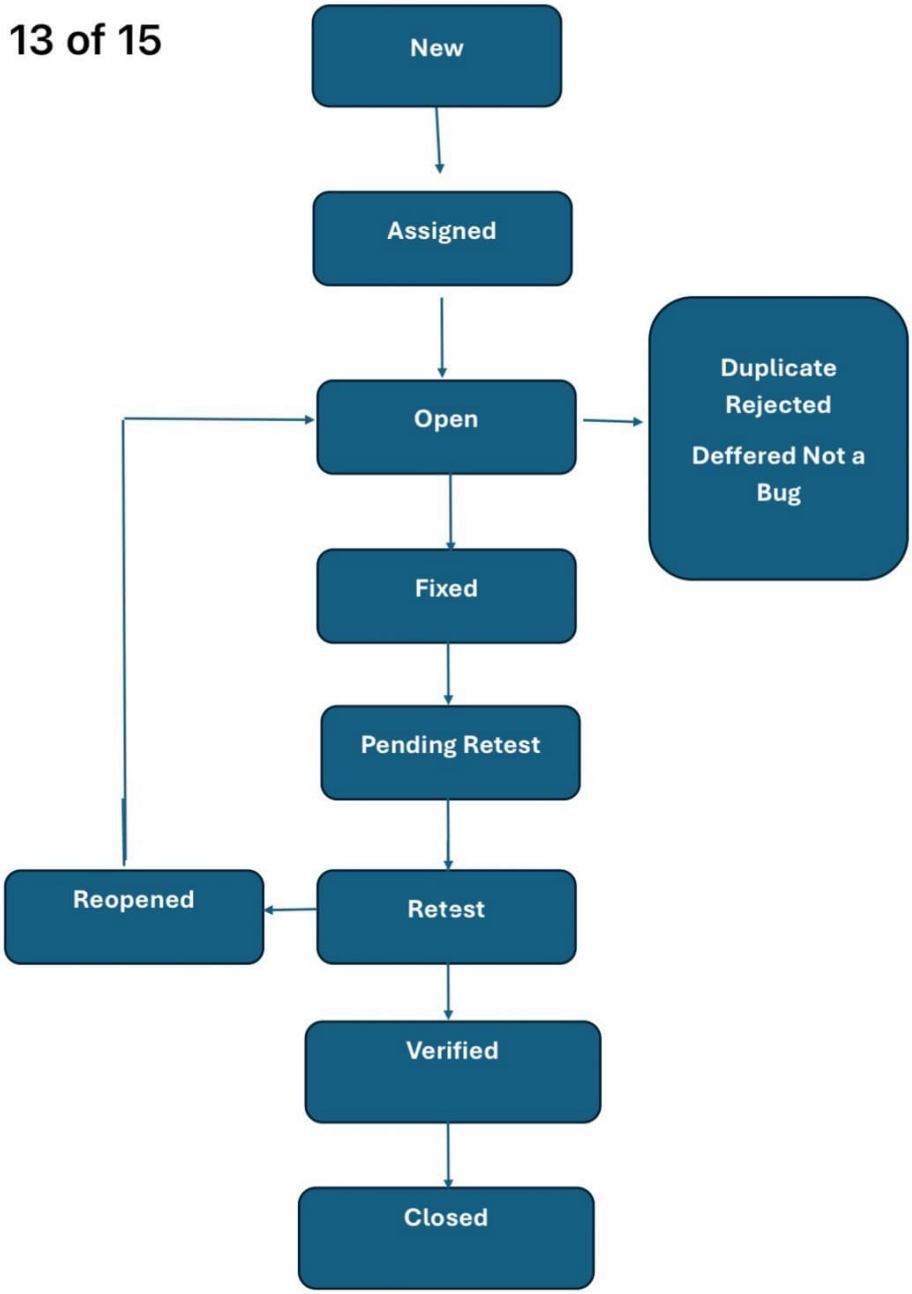
- **Software Development:** To plan work, execute tasks, and manage changing requirements.
- **Scrum Projects:** Useful for managing backlogs, sprints, and sprint planning.
- **Kanban Projects:** Helps visualize work using Kanban boards and improve workflow.
- **Agile Transformation:** Good for teams shifting to Agile with backlog, sprint, and tracking support.
- **Cross-Functional Teams:** Can be used by teams like design, marketing, and operations.
- **Issue & Bug Tracking:** Ideal for projects with frequent issues, bugs, or task updates.
- **Custom Workflows:** Works well for teams that need tailored workflows.
- **Collaboration & Communication:** Allows comments, file attachments, and team coordination.
- **Reporting & Analytics:** Provides useful reports to analyze progress and fix problems quickly.

ACTIVITY – I

DEFEAT / BUG LIFE CYCLE

- **A defect goes through different stages** from the moment it is found until it is fixed and closed.
- **Each stage shows the current status** of the defect so the team knows what is happening with it.
- **This cycle helps teams communicate clearly and work efficiently** to track, assign, fix, and verify bugs.
- **Defect status**
- **Defect status** shows the current stage a bug is in during the defect life cycle.
- It helps the team understand **how far the bug has progressed** toward being fixed.
- The main goal is to **track, communicate, and manage defects clearly and efficiently** throughout the project.

Defect States Workflow



- **New:** The tester finds a bug and reports it for the first time.
- **Assigned:** The test lead reviews it and assigns it to a developer.
- **Open:** The developer starts analyzing and working on the bug.
- **Fixed:** The developer fixes the bug and marks it as fixed.
- **Pending Retest:** The fix is ready, and the tester needs to retest it.
- **Retest:** The tester checks whether the bug is truly fixed.
- **Verified:** The tester confirms that the bug is resolved and working correctly.
- **Reopen:** If the bug is still present, the tester sends it back for fixing.
- **Closed:** The bug is gone, and the tester marks it as closed.
- **Duplicate:** The bug is the same as another existing bug.
- **Rejected:** The developer decides it is not a valid or real bug.
- **Deferred:** The bug is not important now and will be fixed in a future release.
- **Not a Bug:** The reported issue is not actually a defect and does not affect functionality.

- **New:** The tester finds a bug and reports it for the first time.
- **Assigned:** The test lead reviews it and assigns it to a developer.
- **Open:** The developer starts analyzing and working on the bug.
- **Fixed:** The developer fixes the bug and marks it as fixed.
- **Pending Retest:** The fix is ready, and the tester needs to retest it.
- **Retest:** The tester checks whether the bug is truly fixed.
- **Verified:** The tester confirms that the bug is resolved and working correctly.
- **Reopen:** If the bug is still present, the tester sends it back for fixing.
- **Closed:** The bug is gone, and the tester marks it as closed.
- **Duplicate:** The bug is the same as another existing bug.
- **Rejected:** The developer decides it is not a valid or real bug.
- **Deferred:** The bug is not important now and will be fixed in a future release.
- **Not a Bug:** The reported issue is not actually a defect and does not affect functionality.

ACTIVITY – II

IMPACT MATRIX

Severity / Priority	High Priority	Medium Priority	Low Priority
High Severity	• Complete site downtime • Payment gateway failure • Add to Cart not working for all products • Login system completely broken • Checkout failure for all users	• Login button not clickable • Category pages not opening • PDP (Product Detail Page) not loading important details • Search returning server errors • Registration submit button not responding	• Rare crash on some pages • Banner slider freezing occasionally • Product image zoom not working sometimes • Wishlist page loading slowly • Session timeout happening too early
Medium Severity	• "Place Order" button working inconsistently • Wrong prices shown on some product pages • Shipping charges not calculated • Tax not applied for some states • Cart quantity updates failing for some items	• Field validation issues in forms • Incorrect error messages • Pagination not working on category pages • Thumbnail images not switching on some PDPs • Sorting options not applying correctly	• Logical errors in UI (misaligned labels) • Incorrect breadcrumb link for one category • Product tags not showing on PDP • Recently viewed items not loading • Recommendations section incorrect
Low Severity	• Logo missing on some pages • Wrong favicon appearing • Spelling mistake in footer • Broken external link in contact page • Incorrect social media icons	• Field length restrictions not working as expected • Minor alignment issues in category list • Text overflow on Wishlist page • Slight delay in image loading • Hover effects not consistent	• Cosmetic issues (font mismatch) • Slight color variation in buttons • Extra white spacing in homepage sections • Minor CSS glitches on mobile view • Uncentered icons in header/footer